

EEE 4540 Term Project: Conditional GAN for Cup vs Mug Image Generation

Alp Onat Topçam 21070005007, Selma İrem Özdemir 21070001028

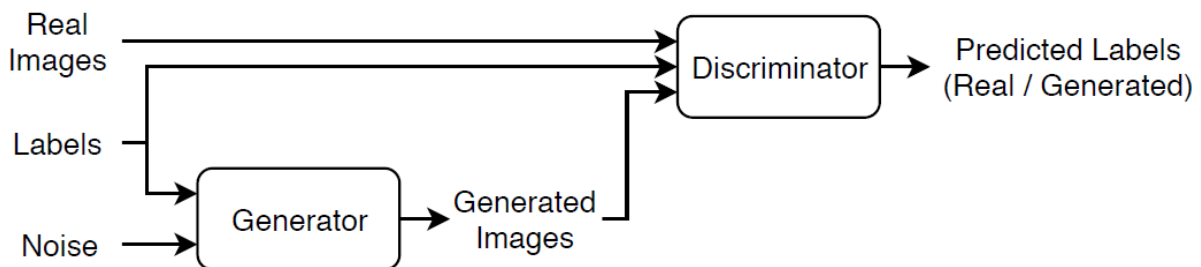
What is CGAN (Conditional Generative Adversarial Network)?

A **Conditional Generative Adversarial Network (CGAN)** is an extended version of the traditional Generative Adversarial Network (GAN), which incorporates **conditional information**, such as class labels or other context, into both the generator and discriminator during training. This structure allows CGANs to generate data that not only looks realistic but also **matches a specific category or condition**.

CGANs consist of two main neural network components:

- **Generator (G):**
Takes a random noise vector z and a condition y (e.g., a class label) as input.
Generates synthetic data $G(z, y)$ that mimics the structure of real data corresponding to that label.
Its objective is to create outputs that are realistic enough to fool the discriminator.
- **Discriminator (D):**
Takes both real samples from the dataset and fake samples from the generator, along with the same condition y .
Tries to classify whether a given sample is real or generated and **whether it correctly matches the given condition**.
Its goal is to distinguish real data from fake data as accurately as possible, while considering the condition.

Thanks to this conditional input, CGANs can be guided to generate samples with **specific attributes**, such as "only dogs", "only grayscale mugs", or "only images of the digit 5".



How CGAN Differs from GAN

While standard GANs generate data **solely from random noise**, CGANs **condition the generation process** on additional input information. This fundamental difference gives CGANs **more control** and enables **targeted data generation**.

Feature	Standard GAN	Conditional GAN (CGAN)
Generator Input	Random noise z	Random noise z + conditional input y
Discriminator Input	Data sample x	Data sample x + conditional input y
Output Control	No control; outputs are random	Controlled output; guided by the input condition
Type of Generated Data	Generic and unstructured	Targeted and class-specific
Use Case	General data synthesis, style generation	Label-specific generation, guided data augmentation

Conditional GANs introduce a powerful mechanism for **guided data generation**. By leveraging class labels or other structured data as input, CGANs can produce synthetic data that aligns with specific user-defined requirements. This makes them highly effective for applications such as **class-conditional image generation**, **data augmentation**, and **controlled content synthesis**. Compared to standard GANs, CGANs offer **greater precision and relevance** in the generation process.

Dataset

The dataset used in this project was collected from multiple sources, including **Kaggle**, **GoogleImageCrawler**, **DuckDuckGo**, and **Bing**. We specifically searched for images of **cups** and **mugs**, and manually **categorized them into two classes** based on their visual characteristics:

- **Colored**
- **Black & White (BW)**

To ensure data quality, all **low-resolution or irrelevant images were filtered out** and removed from the folders during preprocessing.

The final dataset consists of the following image counts:

- **597 images** - Black & White Cup images
- **909 images** - Colored Cup images
- **269 images** - Black & White Mug images
- **931 images** - Colored Mug images

This curated dataset provides a clean and well-balanced structure for training a Conditional GAN (CGAN) that can learn to generate class- and style-specific images.

Problems Encountered in Generating Black & White Cups with CGAN

1. Difficulty in Generating BW Cups Using RGB Output

At the beginning of the project, we attempted to generate Black & White (BW) cups using a CGAN model that was set up to output RGB images. However, this led to two major issues:

a. Mismatch Between Output Format and Target Class (BW Cups)

Although grayscale (BW) images can technically be represented in RGB format by setting all three channels equal (e.g., [128, 128, 128]), in practice this is hard for the generator to learn.

The RGB color space is broad, and grayscale images represent only a narrow subspace within it. The generator often prefers to create colored outputs, since they offer more variety and can more easily fool the discriminator.

As a result, the model struggled to produce truly grayscale outputs, even when conditioned on the "BW Cup" label.

b. Mode Collapse and Class Imbalance

In addition to the format issue, we observed mode collapse during training. The CGAN tended to generate only colored cups, regardless of the input label.

This happened due to several factors:

The dataset was imbalanced:

- 597 BW Cup
- 909 Colored Cup

The generator quickly learned that producing colored cups was sufficient to fool the discriminator, and thus stopped learning to generate BW cups altogether.

As a result, the model ignored the label input, producing similar or repetitive outputs—a clear sign of mode collapse.

Was Expecting RGB Output for BW Cups a Wrong Approach?

Problem	Explanation
BW images in RGB format require equal channel values	Harder for the generator to learn naturally
Generator prefers colorful outputs to fool the discriminator	Grayscale generation is a narrower target
BW classes were underrepresented in the dataset	Increases risk of mode collapse
Generated images <i>looked</i> colored due to normalization	Even if grayscale was learned, visualization showed color due to minor channel variation

Expecting a CGAN with RGB output to generate black-and-white images is **not technically incorrect**, but it is **highly inefficient**. Without careful handling, this approach leads to **mode collapse**, **label neglect**, and poor output diversity.

A grayscale-aware architecture with proper conditioning and class balance is essential for meaningful BW generation.

Solutions:

1. Use Grayscale Output for BW Classes

- Change the generator's output from 3 channels (RGB) to **1 channel (grayscale)**.
- Alternatively, enforce grayscale format by replicating a single value across all channels: [gray, gray, gray].

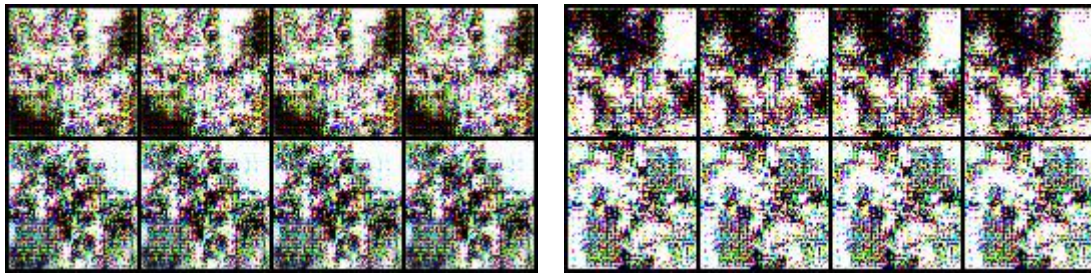
2. Improve Dataset Balance

- Address class imbalance using **oversampling**, **class weights**, or synthetic augmentation for BW classes.

3. Add Anti-Mode-Collapse Techniques

- Techniques such as:
 - **Feature matching**
 - **Mini-batch discrimination**
 - **Spectral normalization**
 - **Gradient penalty (e.g., WGAN-GP)**
can help stabilize training and encourage diversity in generated outputs.

After encountering these issues, we discontinued the cross-referencing of black and white with color data. We have since **transitioned to utilizing the colored cup and mug dataset**.



Model Training Insights and Observations

We experimented with over ten different models. Since most of them were trained to distinguish between black-and-white and colored images, we frequently encountered mode collapse. The training time for each model ranged from approximately 10 to 15 hours. Thanks to our checkpointing strategy, we were able to resume training from where we left off even in the event of internet disconnections.

Due to the limited availability of resources and support for CGANs in MATLAB, we implemented and trained our models in the Google Colab environment instead.

- **Batch Size:** The number of samples processed by the Generator and Discriminator in one forward/backward pass. Larger batches give smoother gradient estimates but use more GPU memory.
- **Image Size:** The height and width (in pixels) of the square images you're training on and generating.
- **nz:** Dimensionality of the latent (noise) vector fed into the Generator. This random vector is what the Generator transforms into an image.
- **Number of Classes:** Number of conditional classes in your dataset (0 = cup, 1 = mug). The model uses this to learn class-specific features.
- **ngf ("number of generator features"):** Base channel count in the Generator's convolutional layers.
- **ndf ("number of discriminator features"):** Base channel count in the Discriminator's layers. Higher values give the network more capacity (but cost more compute).
- **Learn Rate:** Learning rate for both Adam optimizers. Controls how big each weight update is too high can diverge training, too low makes learning very slow.
- **Beta:** The β_1 parameter in Adam's momentum estimate. DCGAN/CGAN recipes often use 0.5 to stabilize training by reducing oscillations.
- **Epochs:** Number of times the training loop goes over the entire dataset.

Model 1



20th Epoch

100th Epoch

500th Epoch

1 Black & White, 0 Colored Mug
0 1 0 1 0 1 0 1

```
image_size = 28
nz = 100
ngf = 32
ndf = 32
nc = 3
num_classes = 2
lr = 0.0002
beta1 = 0.5
num_epochs = 500
batch_size = 64
```

Loss_D	Loss_G
0,9229061093	1,18411521
1,22521208	0,7686664973
1,201320725	0,7465022377

Problems of This Model:

1. **Too-low resolution (28×28)**

Downsampling your photos to 28×28 removes almost all the fine edges and contours that distinguish a black&white mug from a colored mug. At that size the generator has almost no structural cues to latch on to, so it just learns blobs of grey or color.

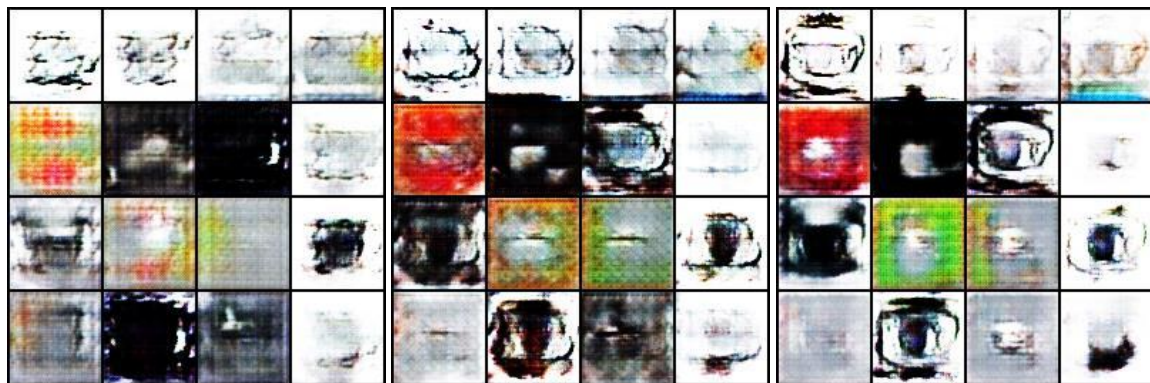
2. **No gradient penalty or spectral norm**

Without something like WGAN-GP, hinge loss, or spectral normalization, there's nothing enforcing a smooth discriminator function. That unbounded critic easily overpowers the generator and mode-collapses follow almost inevitably.

3. **Imbalanced training dynamics**

Even small hyperparameter mismatches (learning rates, betas, batch sizes) can push D and G into very different regimes. (D_loss ≈ 1.2, G_loss ≈ 0.7) show D dominating G has effectively stopped learning.

Model 2



60th Epoch

80th Epoch

100th Epoch

0 Cup, 1 Mug

0*8, 1*8

```
image_size = 32
batch_size = 256
nz          = 100
num_classes = 2
lr          = 1e-4
beta1, beta2= 0.0, 0.9
n_critic    = 5
lambda_gp   = 10.0
epochs      = 1000
```

Loss_D	Loss_G
1.1328	1.2632
1.0074	1.2960
1.0058	1.4358

Problems of This Model:

1. **No regularization on D**

Without any spectral-norm, gradient penalty, label smoothing or instance noise, your Discriminator will “win” almost immediately and starve the Generator of useful gradients.

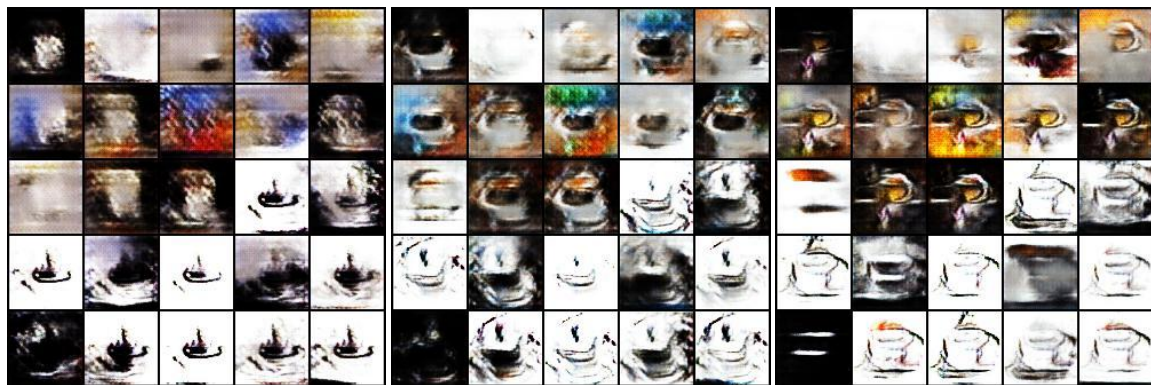
2. **Mode-collapse-prone loss**

The classic binary cross entropy GAN loss is wellknown to be brittle. Once D excels, `loss` goes to zero and G falls into repeating the same few outputs (mode collapse).

3. **Imbalanced training dynamics**

With identical learning rates, no gradient penalty, and no care for the relative strength of D vs G, the Discriminator almost always “outruns” the Generator.

Model 3 (CGAN + LPIPS)



80th Epoch

110th Epoch

150th Epoch

1 Black & White Cup, 0 Colored Mug
13*0, 12*1

```
image_size = 64
batch_size = 64
nz = 100
num_classes = 2
lr = 0.0002
beta1, beta2 = 0.5, 0.999
n_critic = 5
ngf, ndf = 32, 32
epochs = 200
```

Loss_D	Loss_G	LPIPS
12.3964	83.6845	0.5434
10.0680	89.6839	0.5243
19.5464	103.1296	0.4998

Problems of This Model

1. Lack of Regularization in the Discriminator

The Discriminator is trained **without any form of regularization**, such as. This allows the Discriminator to **rapidly dominate** during training, leading to vanishing gradients for the Generator. As a result, the Generator is deprived of meaningful feedback and cannot improve effectively.

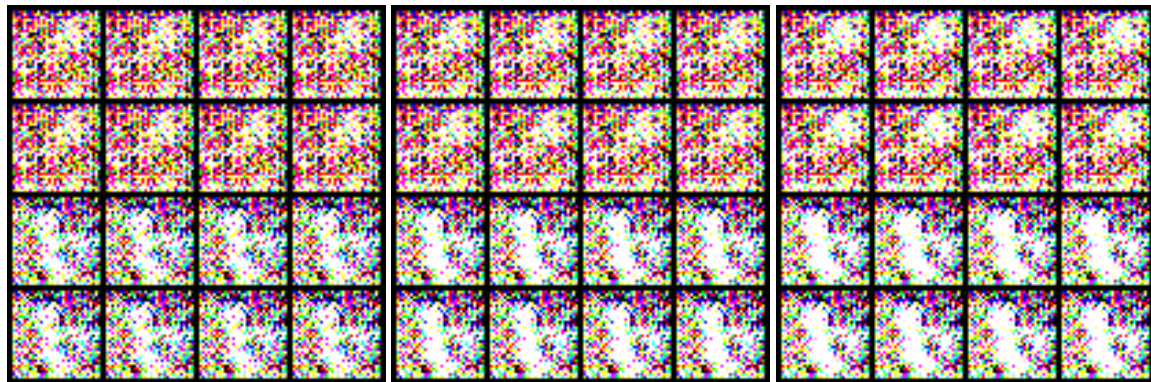
2. Loss Function Prone to Mode Collapse

The model relies on the **standard binary cross-entropy (BCE) GAN loss**, which is known to be **fragile and unstable**. Once the Discriminator becomes strong, the Generator receives near-zero gradients. This often results in **mode collapse**, where the Generator repetitively produces a small subset of outputs, ignoring the diversity of the latent space.

3. Imbalanced Training Dynamics

The training uses **Identical learning rates** for both Generator and Discriminator **No gradient penalty**. **No adjustment for relative model capacity**. These factors cause the Discriminator to **consistently outperform** the Generator, destabilizing the adversarial balance. This imbalance leads to poor Generator learning and oscillating or divergent training behavior.

Model 4



80th Epoch

90th Epoch

100th Epoch

0 Cup, 1 Mug
0*8, 1*8

Loss_D	Loss_G
2.4738	-1.4611
2.4211	-1.4237
2.4468	-1.4461

Problems of This Model

1. Weak conditional signal

The class embedding is only injected once as a tiny 2-D vector at the very first layer, so the cup/mug label information gets drowned out deeper in the network.

The Discriminator never enforces “this is a cup” vs. “this is a mug” (no ACGAN classifier head or projection).

2. Critic–Generator imbalance

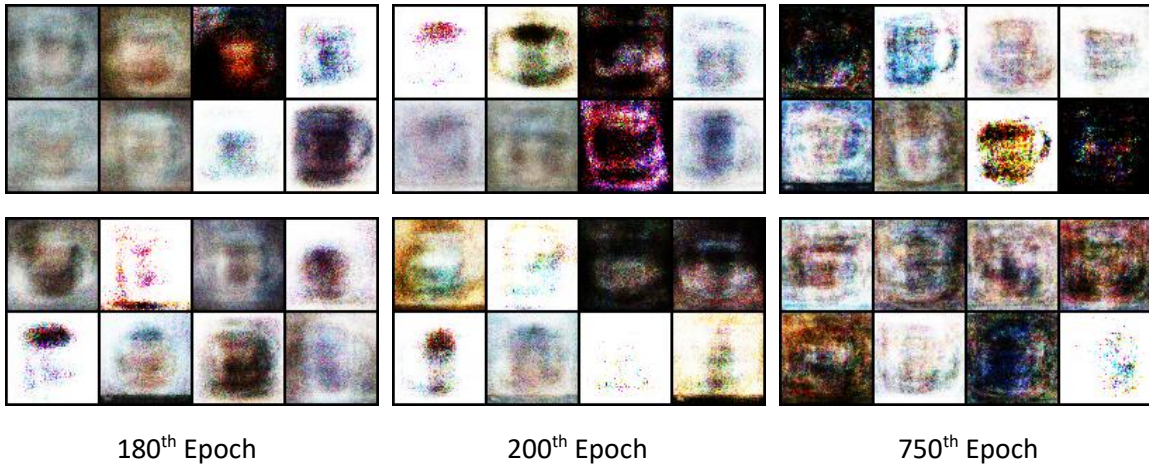
Using `n_critic = 5` gives the Critic too many updates per G update.

Critic loss settles around ~2.4 and Gen loss around ~-1.45 with no further change, indicating both networks are “stuck.”

3. Missing spatial attention

Unlike previous Self-Attention versions, this “fast” model drops any attention mechanism, so it cannot learn long-range spatial dependencies that help form coherent objects.

Model 5



0 Cup, 1 Mug

8*0, 8*1

First 8 = Cup, Second 8 = Mug

```
image_size = 64
batch_size = 64
nz = 100
num_classes = 2
lr = 0.0002
beta1, beta2 = 0.5, 0.999
n_critic = 5
ngf, ndf = 32, 32
epochs = 880
```

Loss_D	Loss_G
1.2354	1.1546
1.3634	1.1392
0.8891	1.7073

Problems of this Model

1. Mode Collapse Risk

The Generator loss stays in the 1.6–2.0 range, with little variation across epochs.

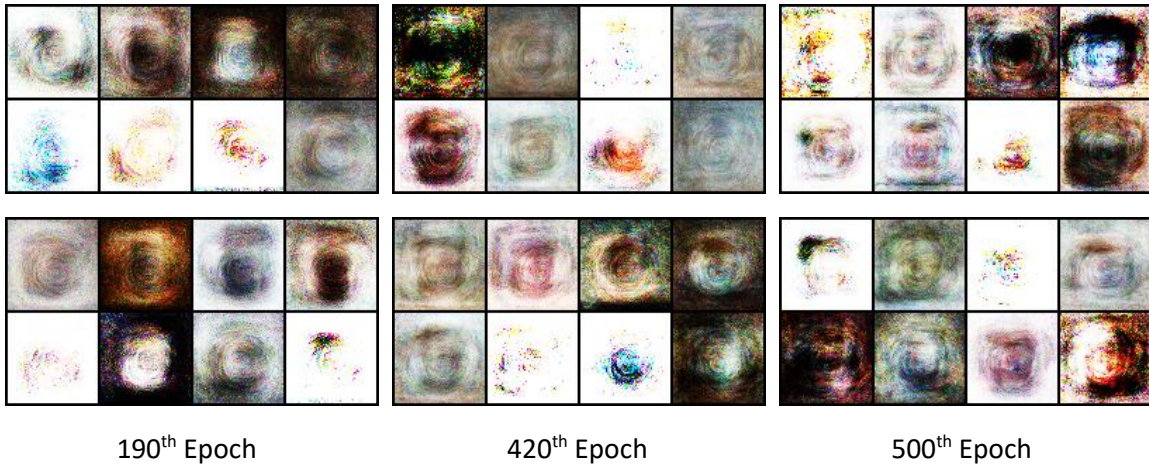
This suggests mode collapse, where the Generator produces similar or identical outputs regardless of input noise.

2. No Evaluation Metrics (FID, LPIPS, etc.)

The training loop tracks only Loss_D and Loss_G.

Without perceptual or distributional metrics, it's hard to objectively assess image quality or diversity.

Model 6 (CGAN + LPIPS)



0 Cup, 1 Mug

8*0, 8*1

First 8 = Cup, Second 8 = Mug

```
image_size = 64
batch_size = 64
nz = 100
num_classes = 2
lr = 0.0002
beta1, beta2 = 0.5, 0.999
LPIPS Lambda = 0.1
epochs = 580
```

Loss_D	Loss_G	Batch LPIPS	Epoch LPIPS
1.3270	1.1564	0.5599	0.5484
1.1864	1.4861	0.5388	0.5329
1.1024	1.4306	0.5359	0.5292

Problems of this Model:

1. No Discriminator Regularization

The Discriminator is trained **without any regularization** (e.g., spectral norm, gradient penalty, or noise injection). This may cause unstable or oscillatory training, Allow D to overpower G (e.g., Loss_D remains near 1.0). Prevent G from receiving smooth gradients.

2. Flat LPIPS Values Indicate Low Semantic Diversity

Despite using LPIPS, the average LPIPS values remain in a narrow band (≈ 0.51 – 0.55) over 12 epochs. This could indicate: The Generator produces semantically similar outputs.

3. MLP Architecture Lacks Spatial Awareness

Using nn.Linear layers means images are flattened, causing the model to ignore spatial context and structure. This limits: Fine texture reproduction, Alignment between object parts.

4. LPIPS Loss Is Not Aligned

The LPIPS is calculated between randomly matched real-fake image pairs, not semantically or label-aligned samples. This weakens LPIPS supervision.

Model 7 (Hinge Loss)



20th Epoch

170th Epoch

330th Epoch

0 Cup, 1 Mug
8*0, 8*1

```
image_size = 32
batch_size = 256
nz          = 100
num_classes = 2
lr          = 1e-4
beta1, beta2= 0.0, 0.9
n_critic    = 5
lambda_gp   = 10.0
epochs      = 1000
```

critic_loss	gen_loss
-5.12	-131.8124
-2.4188	-1704.9083
-1.9238	-4388.0391

What is Hinge Loss?

Hinge loss is a type of margin-based loss function originally used in Support Vector Machines (SVMs). In the context of **GANs**, especially **Conditional GANs (CGANs)**, hinge loss helps improve training stability and image quality by enforcing a margin between real and fake data scores.

Instead of predicting probabilities (like in binary cross-entropy), the discriminator (called the **critic** in this setup) produces real-valued scores, and hinge loss pushes:

- Real samples to be scored **greater than or equal to 1**
- Fake samples to be scored **less than or equal to -1**

This margin-based enforcement improves the learning dynamics and prevents vanishing gradients.

critic_loss (Critic / Discriminator Loss)

The critic_loss measures how well the **critic** (also known as the discriminator) distinguishes between real and generated (fake) images.

It uses **hinge loss** to enforce a margin:

- Real image–condition pairs should receive scores ≥ 1
- Fake image–condition pairs should receive scores ≤ -1

The ReLU ensures that only scores violating the margin contribute to the loss.

A lower critic_loss indicates the critic is doing a good job separating real and fake examples.

gen_loss (Generator Loss)

The gen_loss evaluates how well the **generator** is able to fool the critic.

It tries to **maximize** the critic's score on fake image–condition pairs, encouraging the generator to produce more realistic outputs

- The generator is penalized if the critic assigns low scores to fake images.
- A lower gen_loss suggests the generator is successfully fooling the critic (producing realistic images).

REFERENCES

- <https://www.geeksforgeeks.org/deep-learning/conditional-generative-adversarial-network/>
- <https://www.coursera.org/articles/conditional-generative-adversarial-network>
- <https://www.mathworks.com/help/deeplearning/ug/train-conditional-generative-adversarial-network.html>

ADDITIONAL FILES

- Codes (Model architectures)
- Images of epoch outputs